

Navigating WWW

Software System Development
Monsoon 2026

Markup Vs Markdown

A markup language is any system where you embed instructions within plain text to tell a machine what that text means or how it should look. It does not dictate *how* you write those instructions.

* All Tag based languages are Markup languages.

Examples: HTML, XML, LaTeX, AsciiDoc, YAML, RTF.

Markdown is a lightweight markup language created by John Gruber in 2004. Its core philosophy is **readability**: a Markdown document should be publishable as-is, as plain text, without looking like it has been marked up with tags or formatting instructions.

Examples: commonmark, GitHub Flavored Markdown etc.

Editors: Dillinger, markdownlivepreview

```
<!DOCTYPE html>
<html>
<head>
  <title>My Website</title>
</head>
<body> <h1>Welcome to DOM</h1>
<p>This is my <a href="https://sai11101989.github.io"
target="_blank">website.</a></p>
</body>
</html>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
<book category="fiction">
<title>The Great Gatsby</title>
<author>F. Scott Fitzgerald</author>
<year>1925</year> <price>10.99</price>
</book>
</bookstore>
```

1. Core Text Formatting

You can format inline text by wrapping it in standard symbols.

```
*   **Bold:** `**text**` becomes text
*   **Italic:** `text` becomes text
*   **Strikethrough:** `text` becomes text
*   **Inline Code:** `text` (used for variables or short commands)
*   **Links:** `[Link Text] (https://example.com)`
*   **Images:** `![Alt text for accessibility] (image.jpg)`
```

Data Interexchange Format

Designed purely to store, structure, and transport data, typically between a server and a web application. It uses simple key-value pairs and arrays, and it does not dictate how content should be displayed or formatted visually.

JSON (JavaScript Object Notation): As mentioned, this is currently the most popular format for transmitting data between a server and a web application. It is lightweight, easy for machines to parse, and relatively easy for humans to read.

```
{  
  "book": "The Great Gatsby",  
  "author": "F. Scott Fitzgerald",  
  "year": 1925  
}
```

YAML (YAML Ain't Markup Language): YAML is designed to be exceptionally human-readable. It relies on indentation (spaces) to define structure rather than brackets or tags. It is heavily used for configuration files (like in Docker or Kubernetes) and data exchange where human readability is the top priority.

```
book: The Great Gatsby  
author: F. Scott Fitzgerald  
year: 1925
```

CSV (Comma-Separated Values): CSV is one of the oldest and simplest data-interchange formats. It is used specifically for flat, tabular data (like spreadsheets or databases). Each line is a data record, and fields are separated by commas. While not good for nested or complex data, it is incredibly efficient for large lists.

```
book,author,year  
The Great Gatsby,F. Scott Fitzgerald,1925
```

DOM

The **Document Object Model (DOM)** is a cross-platform and language-independent programming interface (API) that represents HTML and XML documents as a logical tree structure.

It acts as a bridge, connecting web pages to programming languages like JavaScript* so that a document's structure, style, and content can be dynamically changed.

Root Node

The document itself acts as the topmost root node of the tree

Element Node

Every HTML element (such as `<p>`, `<div>`, or `<h1>`) becomes an individual element node within the tree.

Text Node

The actual text content inside those HTML elements is represented as separate text nodes. Because text nodes do not have child nodes, they are always "leaf nodes" at the end of a branch.

Attribute Note

HTML attributes are also represented as nodes

```
<!DOCTYPE html>
<html>
<head>
  <title>My Website</title>
</head>
<body> <h1>Welcome to DOM</h1>
<p>This is my <a href="https://sai11101989.github.io"
target="_blank">website.</a></p>
</body>
</html>
```

- `<html>` (root node)
 - `<head>` (Element node)
 - `<title>` (Element node)
 - "My Website" (Text node)
 - `<body>` (Element node)
 - `<h1>` (Element node)
 - "Welcome to DOM" (Text node)
 - `<a>` (Element node)
 - href (attribute node)
 - `<p>` (Element node)
 - "This is my website." (Text node)

*We shall look at how DOM works with JavaScript during our JavaScript Sessions

Browser

A web browser is a piece of software that fetches resources from a server and translates raw code (HTML, CSS, JavaScript) into an interactive graphical interface.

When you type a URL, several distinct layers of the browser spring into action:

User Interface

The address bar, back/forward buttons, and bookmarks menu. Everything except the window where the web page itself is displayed.

Browser Engine

The bridge that marshals actions between the UI and the rendering engine.

Networking Layer

Handles network calls like HTTP requests, managing CORS (Cross-Origin Resource Sharing), and caching.

Script Engine

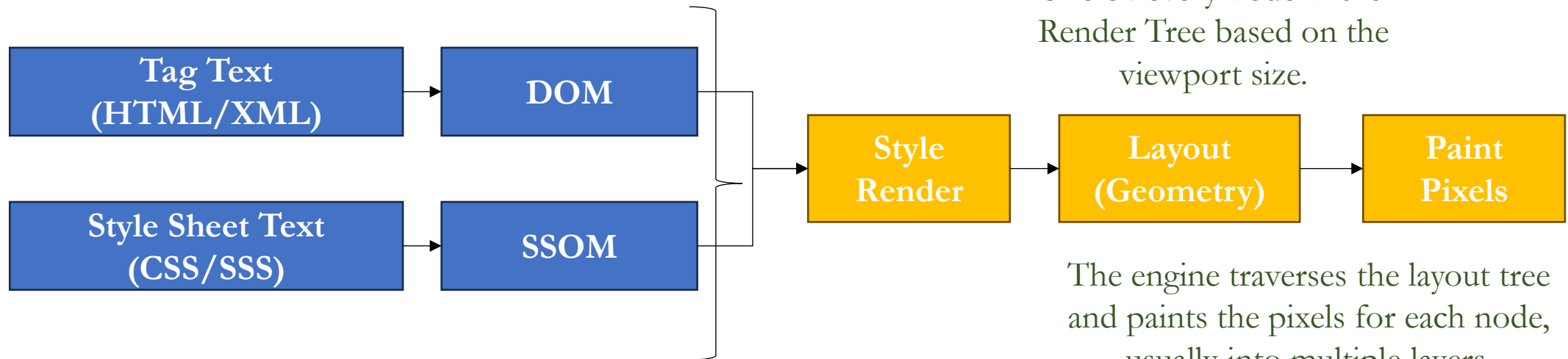
Parses and executes JavaScript code (e.g., V8 in Chrome, SpiderMonkey in Firefox).

Rendering Engine

The subsystem responsible for taking HTML and CSS and painting it onto the screen (e.g., Blink in Chrome, WebKit in Safari).

Rendering Engine

The browser parses HTML markup to construct the Document Object Model (DOM) tree. Simultaneously, it parses CSS to build the CSS Object Model (CSSOM) tree.



The engine calculates the exact geometric position and size of every node in the Render Tree based on the viewport size.

The engine traverses the layout tree and paints the pixels for each node, usually into multiple layers.

The DOM and CSSOM are combined into a Render Tree, which contains only the nodes required to render the page (excluding hidden elements like <head> or display: none).

The browser sends these painted layers to the GPU, which composites them together to display the final image on the screen.

Rendering Benchmarks – (2D Web)

Page Load : a benchmark is defined by Google's **Core Web Vitals***. These metrics measure how fast a user perceives the page to load, rather than when the entire DOM is technically finished.

First Contentful Paint (FCP)

The time it takes for the browser to render the very first piece of DOM content (text or image). The ideal benchmark is **under 1.8 seconds**.

Largest Contentful Paint (LCP)

The time it takes for the largest visible element (usually a hero image or a massive text block) to render fully. The ideal benchmark is **under 2.5 seconds**.

Interaction to Next Paint (INP)

Once the page is loaded, this measures how quickly the browser responds to a user's click or tap and paints the next frame. The ideal benchmark is **under 0.2 seconds (200 milliseconds)**

*A web application exceeds these benchmarks, it is generally considered a signal of mounting technical debt—often caused by bloated DOM trees or synchronous JavaScript blocking the rendering pipeline.

Rendering Benchmarks – (3D Web)

When rendering interactive 3D environments, physics simulations, or utilizing WebXR APIs, "**render time**" shifts from a one-time page load metric to a strict per-frame execution budget. In these environments, rendering takes place in a continuous loop governed by `requestAnimationFrame`.

Per-Frame Budget

To achieve a smooth 60 Frames Per Second (FPS), the browser's JavaScript engine and rendering pipeline must execute all logic and WebGL draw calls in **under 16.6 milliseconds (0.016 seconds)** per frame.

Immersive Hardware Budgets

If the browser content is being projected to specialized displays that require 90 FPS, that per-frame execution budget drops to **~11 milliseconds (0.011 seconds)**.

Motion-to-Photon Latency

It measures the total time from when a user physically moves their head to when the browser updates the pixels on the display to reflect that movement. The industry standard mandates that this entire process stays **under 20 milliseconds (0.02 seconds)** to prevent user discomfort or simulator sickness.

*A web application exceeds these benchmarks, it is generally considered a signal of mounting technical debt—often caused by bloated DOM trees or synchronous JavaScript blocking the rendering pipeline.

HTTP vs HTTPS

Feature	HTTP	HTTPS
Full Name	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Security	Insecure. Data is sent as raw, readable text.	Secure. Data is scrambled into ciphertext.
Encryption Protocol	None	SSL (Secure Sockets Layer) / TLS (Transport Layer Security)
Default Port	Port 80	Port 443
Data Integrity	Vulnerable to "Man-in-the-Middle" (MitM) attacks; data can be altered in transit.	Cryptographically verified; data cannot be tampered with undetected.
Authentication	No validation of the server's identity.	Uses digital certificates to prove the server is who it claims to be.
SEO Impact	Flagged as "Not Secure" by browsers; penalized in search rankings.	Required for modern web standards; boosted in search rankings.

When you send data over standard **HTTP**, your information (like a password or an API payload) is sent across the internet in plain text. Anyone monitoring the network traffic—such as an internet service provider or a malicious actor on a public Wi-Fi network—can intercept and read that data perfectly.

When you use **HTTPS**, the browser and the server perform a "TLS Handshake" before any data is sent. They agree on a secret cryptographic key. Once established, all data sent between the browser and the server is encrypted using that key. If a malicious actor intercepts the traffic, all they see is a random string of useless characters.

SSL & TLS

They are both cryptographic protocols designed to do the exact same thing (encrypt data between a browser and a server), but SSL is entirely obsolete.

To verify a server's identity, SSL/TLS relies on a system called **Public Key Infrastructure (PKI)** and entities known as **Certificate Authorities (CAs)**.

The process is essentially a digital version of checking a passport: your browser doesn't inherently trust the server, but it trusts the authority that issued the server's "ID card."

Internet Engineering Task Force



<https://www.ietf.org/>

